

Binary Hacking: Cheats for Game Boy Wars 2

Thomas Hetebrüg¹ Leah Ludwig¹ Julius Hamel¹

¹Heidelberg University

Motivation and Challenges

We chose the GameBoy Color game Game Boy Wars 2 (GBW2) as a topic for the practical. Our goal was get an understanding of key game mechanics and to write cheats based on the mechanics we identified.

We specifically chose GBW2 as a relatively unknown game with little available sources to start this practical from scratch. The turn based nature of the strategy game GBW2 gave us an easy entryway analyze the memory state in the beginning, but also made it quite challenging to keep track of instructions and memory blocks being used by both players.

Methodology and Concept

In the beginning while having no grasp for the game loop we simply started by searching the RAM memory for values that correlate with a value of interest in the game, for example a units HP values. We would then verify and investigate these potential memory addresses by setting read and write breakpoints in an emulator and looking how and when these values were modified in detail.

After gaining more knowledge about the structure of GBW2, we were also able to identify the us and structure of memory blocks more easily. One good example for this are the memory addresses of structures, which comes right after the RAM block of combat units. After learning and understanding about the units memory we were able to understand the concept of the structure memory much faster.

We wrote a python framework based on the emulator pyboy, which also has a memory manipulation API, with can be used to run GBW2 and select wanted cheats for either team from a graphical interface.

Most of our cheats in this framework are based on pyboy hooks, which trigger when the programm counter (PC) reaches a specific instruction. We chose to go for this approach instead of static assembly manipulation to make it easier to enable a cheat for a single team.

While there are a few functions and procedures which get called in the same team order every time where we could have injected our own modified assembly functions at other times it is the exact same instruction being called for both teams, that loads different values based on addresses in the HL registers. Since we were unable to effectively handle such cases we opted for pyboy hooks and high-level functions, such as if-else, to check if a cheat should be applied or not.

Unit and Structure memory

After buying a unit their stats get saved in 16-byte blocks in the memory addresses C980 to CE9F, with the units of the red team being saved in C980-CC09 and CC10-CE9F containing the white units.

C980:	00 03 08 03	00 00 32 00	63 63 49 00	28 00 00 00
C990:	7F 7F 7F 7F	7F 7F 7F 7F	7F 7F 7F 7F	7F 7F 7F 7F
C9A0:	7F 7F 7F 7F	7F 7F 7F 7F	7F 7F 7F 7F	7F 7F 7F 7F

Figure 1. A red 'Infantry' in memory

The first byte depicts the unit type. There are 11 different unit types with red units being represented by even and white units by odd numbers. The second and third byte represent the column and the row of the tile the unit is standing on. Additionally the tile type is saved in the seventh byte. The table below shows some of the possible tile types.

Tile type	Byte
Red Commercial	32
White Commercial	33
Grass	3D
Forest	3F
Mountain	40

The fourth byte depicts the units 'exhaustion' state. Units are only allowed one move per turn. If a unit is allowed to move this byte is 02, if it has already moved or has just been recruited this turn it is 03.

The third block of bytes contains a units stats, with the ninth byte saving the units HP, the tenth byte representing the ammunition of a unit, which gets decreased through movement or combats and bytes eleven and twelve being weapon stats, which are used to calculate the combat damage.

The last four bytes seem to be static for all units of the different teams.

The memory addresses containing the stats for structures are located right behind the unit memory addresses, starting at address CEAO. The stats for one structure is represented in a four byte block.

CEA0:	3A 07 04 14	32 02 06 14	32 03 06 14	3B 06 06 14
CEB0:	3B 08 06 14	33 0B 06 14	33 0C 06 14	32 02 07 14

Figure 2. Eight structures on map 'Devilman' in memory

The first byte represents the structure type. The table below shows all tile types of the map 'Devilman'.

Structure Type	Byte
Red Command Base	3A
White Command Base	3B
Red Commercial	32
White Commercial	33
Red Industrial	30
White Industrial	31

The second and third byte are the tile coordinates, similar as in unit memory. The fourth byte contains the structure HP, which is divided by 10 compared to its displayed value.

Combat Loop

Whenever a unit is selected, their memory is copied from the memory addresses described above to another section in the RAM, starting at C4D3. Once another unit is selected to be attacked their memory is copied to C517 - C526. When combat is initialized, the game calculates the damage for both units and updates the previously mentioned RAM sections. This can be described with the following equations, where unitTypeValue and tileTypeValue are loaded from the ROM, depending on the unit type and tile type.

$$\text{newHP} = \text{currentHP} - (\text{baseDamage} - \text{damageReduction}) \quad (1)$$

$$\text{baseDamage} = \left\lfloor \frac{\text{unitTypeValue} * \text{opposingUnitHP}}{99} \right\rfloor + \begin{cases} 5 & \text{for attacked team} \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

$$\text{damageReduction} = \left\lfloor \frac{\text{tileTypeValue} * \text{baseDamage}}{100} \right\rfloor \quad (3)$$

The current HP for the red team get loaded into C60C and the new HP gets loaded into C611. The white team memory addresses are C614 for the current HP and C619 for the new HP. These values then get transformed into the displayed HP in the range 1-10, with the following calculation:

$$\text{displayHP} = \left\lfloor \frac{\text{memoryHP}}{10} \right\rfloor + 1 \quad (4)$$

The game then loops the combat animation for both teams, decreasing the current HP until it is the same as the new HP. For every decrease one individual of the unit dies. After the combat is done, the units new stats get loaded back to their base memory block.

Results

As mentioned earlier, we created a framework to run the game and select the cheats through a GUI.



Figure 3. A screenshot of our framework

The full list of cheats we achieved to write is:

1. Invincibility
Our Invincibility cheat makes units invincible. They do not lose any HP in combat and no units die in combat animation.
2. Money Locking
When Money Locking is active a teams money does not get decreased after buying a new unit.
3. Infinite Moves
The Infinite Moves cheat allows units to move multiple times in one turn, this includes attacking multiple times.
4. Instant Capture
Instant Capture makes units capture enemies structures in one turn, even if they have more HP than the units structure damage.
5. One-Hit Kill
The One-Hit Kill cheat is the opposite of invincibility and makes units kill every enemy unit in one attack.
6. Money Manipulation
Our Money Manipulation cheats allows the player to manually increase or decrease their own or the enemies money by an arbitrary input or pre-set amount.



Figure 4. A screenshot of the Money Manipulation GUI

Future Work

For future work it would be very interesting to analyze how the computer controls a player and makes gameplay decisions. Additionally attempting to add new functions in free memory space or with python API support, like 'capturing' defeated enemy units by transferring their memory into your teams unit memory would be a nice idea.